

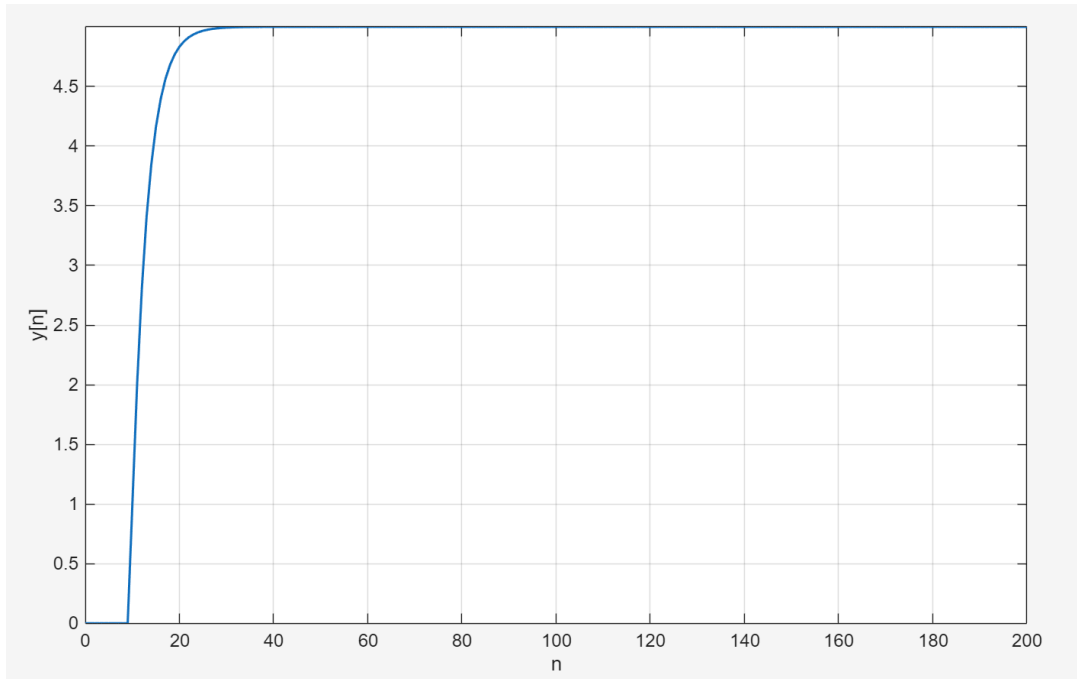
EE110B Lab#3

Nathan Taing

Evaluating stability of second order systems

a) Difference equation: $y[n - 1] - c_0 y[n - 2] + x[n] = y[n]$

b)



This plots the output of $x[n] = u[n-10]$ and the differential equation cited in question a. The code to achieve this plot is detailed below.

Code:

% The next two lines make your plots look better

```
set(0, 'DefaultAxesFontSize', 12);
```

```
set(0, 'DefaultLineLineWidth', 1.5);
```

```
n_arr = 0:200;
```

```
c_O = 0.2;
```

```
x_arr = [zeros(1,10), ones(1, length(n_arr)-10)];
```

```
y_arr = zeros(1, length(n_arr));
```

```
for idx = 1:length(n_arr)
```

```
    if idx-1 >= 1
```

```
        y_prev1 = y_arr(idx-1);
```

```
    else
```

```
        y_prev1 = 0;
```

```
    end
```

```
    if idx-2 >= 1
```

```
        y_prev2 = y_arr(idx-2);
```

```
    else
```

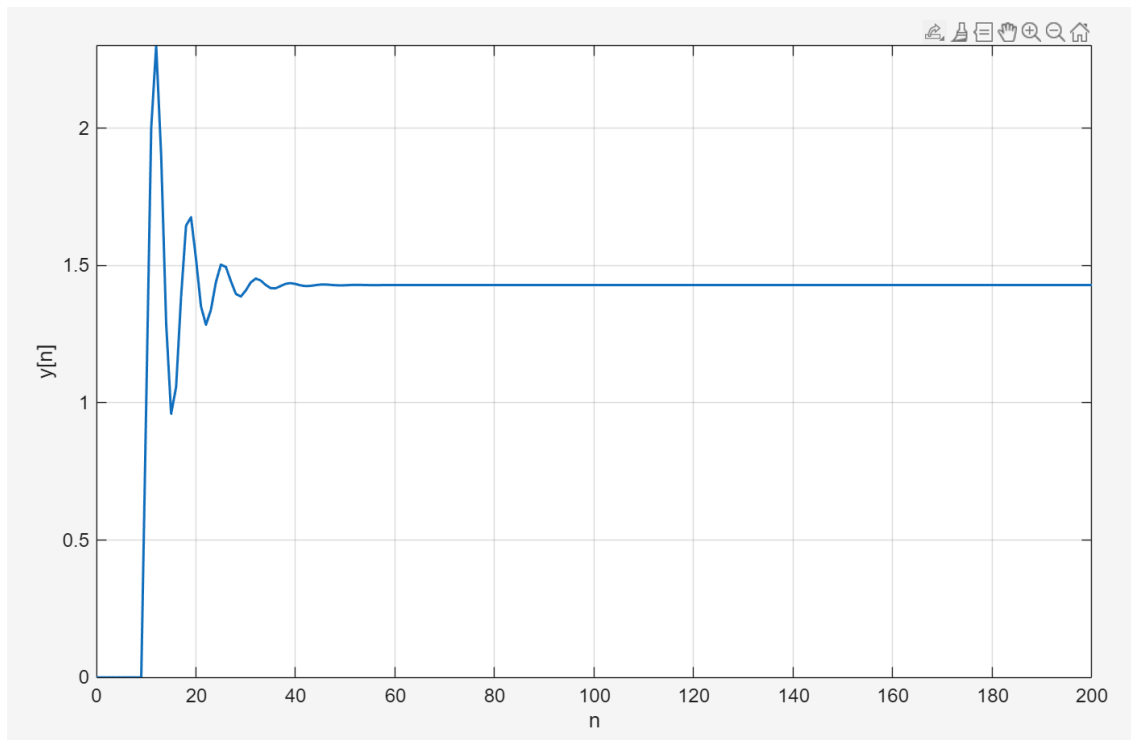
```

    y_prev2 = 0;
end
y_arr(idx) = x_arr(idx) + y_prev1 - c_O*y_prev2;
end
plot(n_arr, y_arr);
xlabel('n');
ylabel('y[n]');
%% The next two lines will make your plot look better
axis tight;
grid on;

```

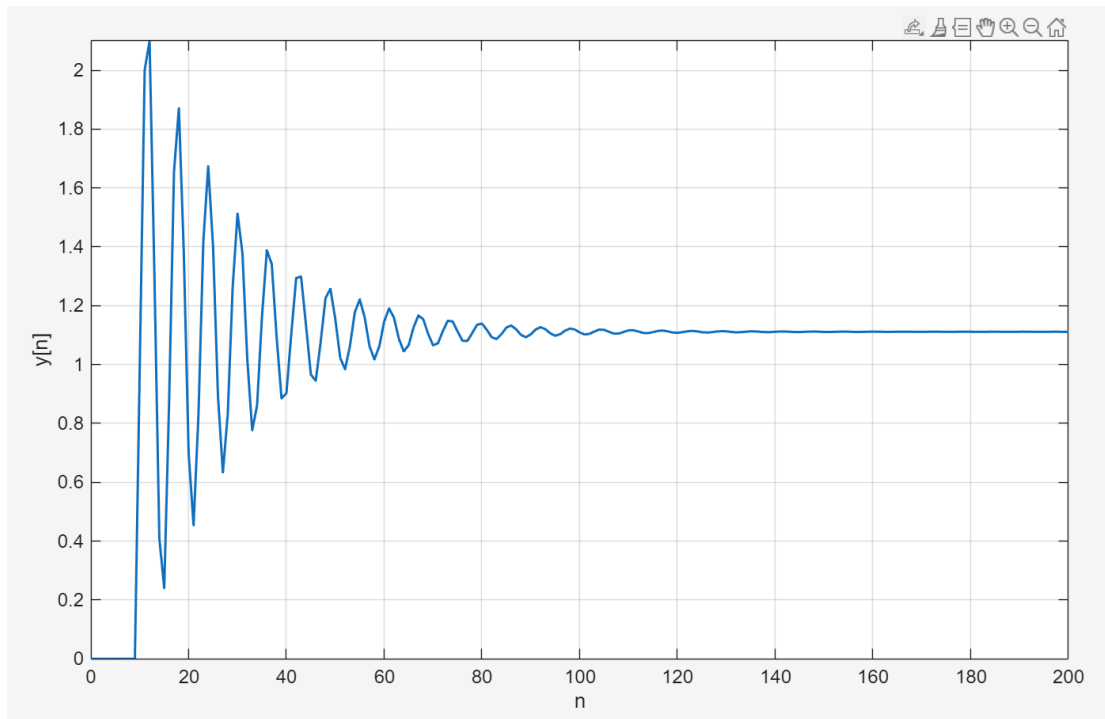
- c) Code utilized is the same as above. We are simply changing the value of c_O .

$$y[n - 1] - 0.7y[n - 2] + x[n] = y[n]$$



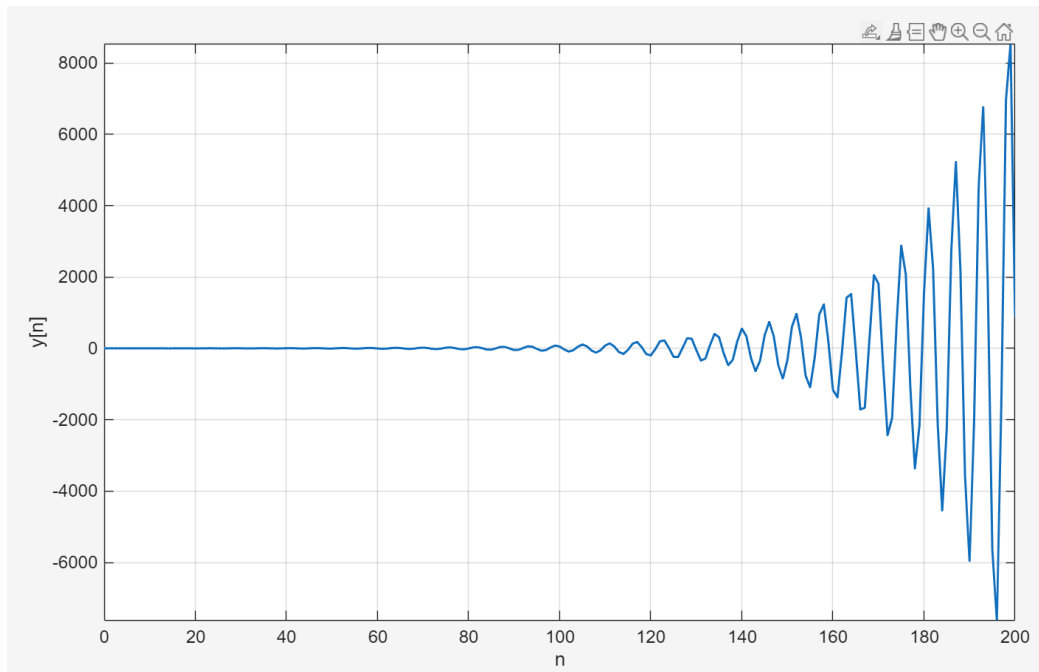
The system oscillates for a brief number of ns. Eventually, it settles to what looks to be about 1.47. The system eventually converges to a constant after initially oscillating. Compared to part b, the maximum value has decreased, we will see if this pattern continues with increasing COs.

$$y[n - 1] - 0.9y[n - 2] + x[n] = y[n]$$



The system oscillates for a greater amount of time compared to the previous test. The settling time is increasing. The system, however, is still stable as it eventually converges to a single value. This value appears to be around 1.1. The trend of decreasing maximum values continues to be followed in this plot.

$$y[n - 1] - 1.1y[n - 2] + x[n] = y[n]$$

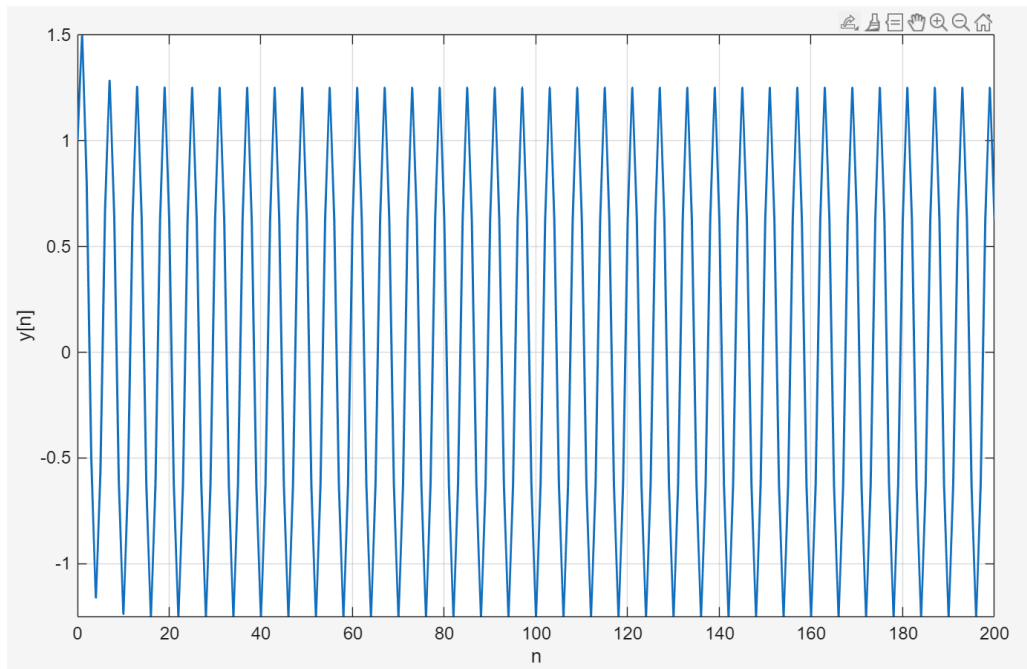


This system will oscillate forever. We can see from the values on the y-axis that the system is increasing quickly and will never converge to a single value. It is no longer a stable system and the maximum value is significantly higher than the previous documented values.

d) $x[n] = \cos(\frac{\pi n}{3})$

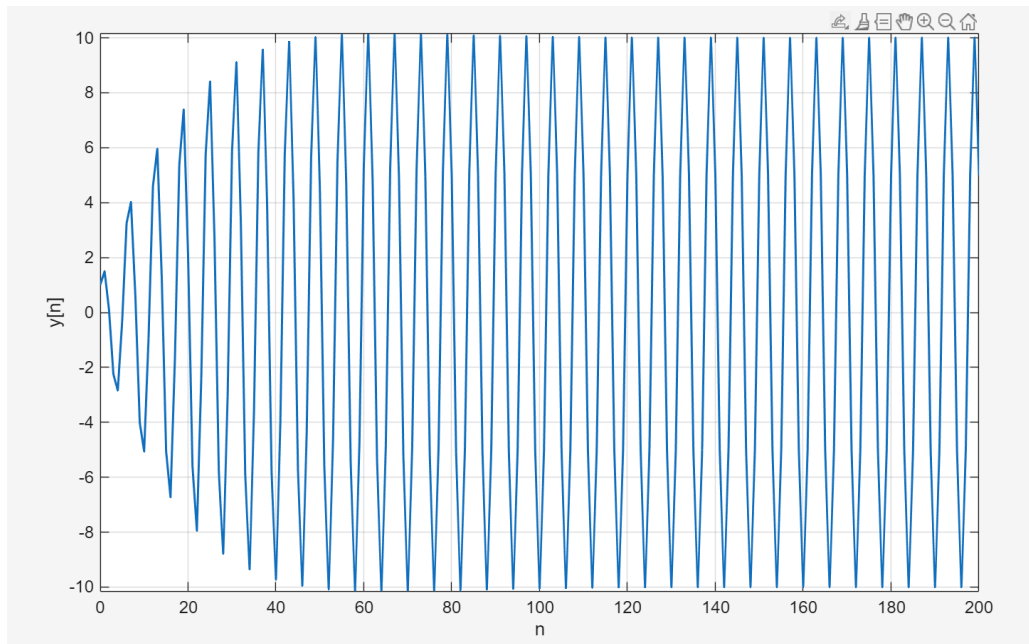
Code utilized is the same as above. We are simply changing the value of `c_O` and we changed the value of `x_arr` to `cos((pi* n_arr)/3)`.

$$y[n - 1] - 0.2y[n - 2] + x[n] = y[n]$$



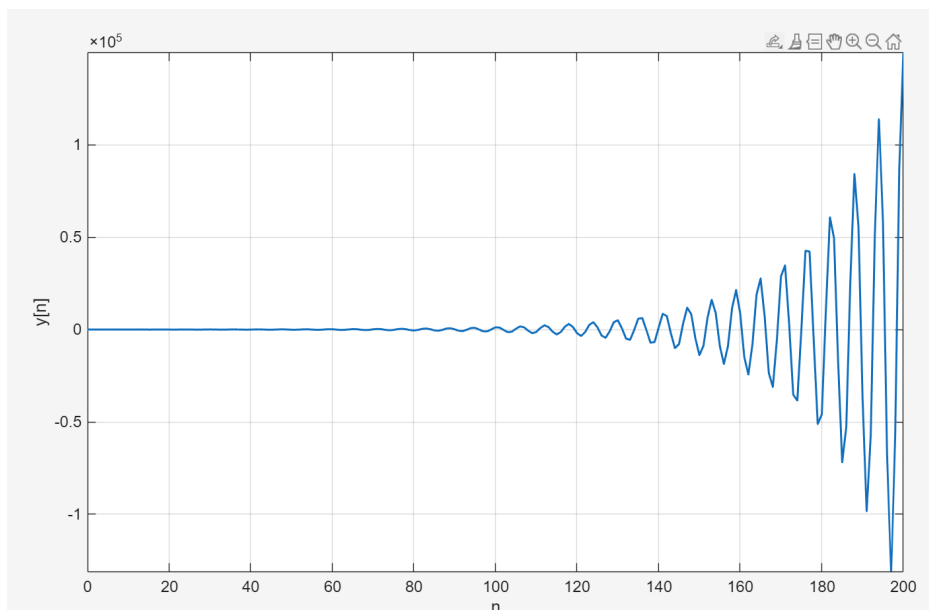
The maximum value recorded is 1.5. This seems to be only a spike as the system seems to enter a stable oscillation after the first few n counts. There is no blow up or crazy values being produced from the plot

$$y[n - 1] - 0.9y[n - 2] + x[n] = y[n]$$



The oscillations first start small, but they grow to about an amplitude of 10. This system also takes a longer time to reach a stable state where the oscillations appear repeatable and constant. We can tell from the y-axis that the maximum value is growing with a bigger CO.

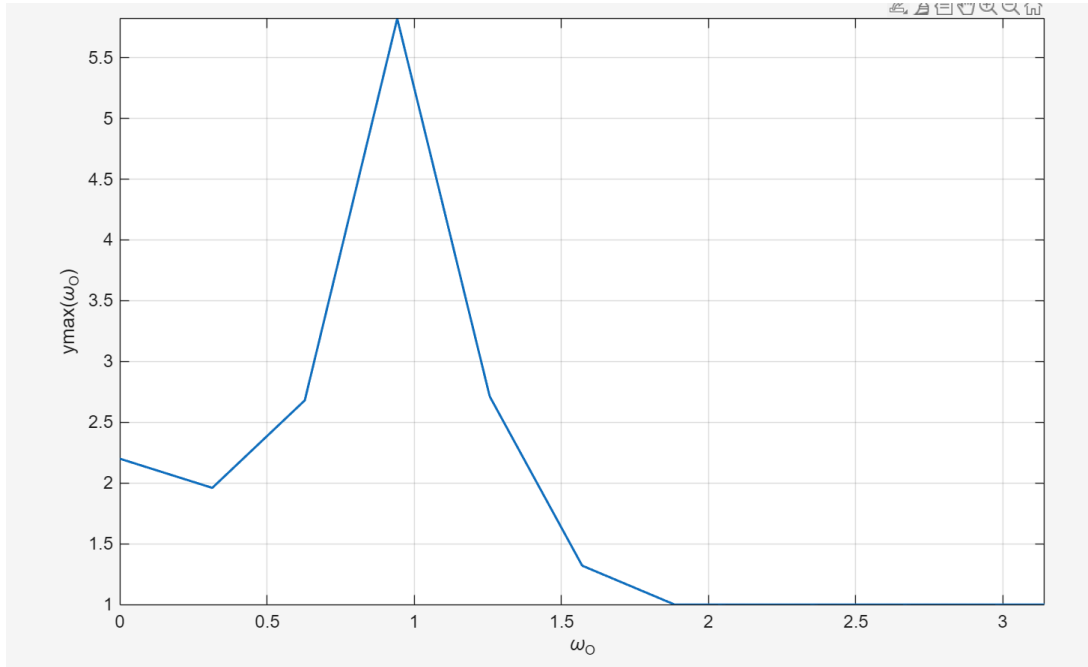
$$y[n - 1] - 1.1y[n - 2] + x[n] = y[n]$$



This system becomes unstable as we can see the oscillations growing and never finding a stable pattern. If you look at the y-axis the values are significantly smaller than the recorded values from the previous attempts. However, since the system is unstable, the

values will eventually blow up to become greater than the maximum values read from the previous figures.

e) $y[n - 1] - 0.8y[n - 2] + x[n] = y[n]$, $x[n] = \cos(\omega_0 n)$



Code:

% The next two lines make your plots look better

```
set(0, 'DefaultAxesFontSize', 12);
```

```
set(0, 'DefaultLineLineWidth', 1.5);
```

```
n_arr = 0:200;
```

```
c_O = 0.8;
```

```
omega_O_arr = linspace(0, 1, 11) * pi;
```

```
max_arr = zeros(1, length(omega_O_arr));
```

```
x_arr = cos(omega_O_arr);
```

```
y_arr = zeros(1, length(n_arr));
```

```
for m_idx = 1:length(omega_O_arr)
```

```
    x_arr = cos(n_arr * omega_O_arr(m_idx));
```

```
    y_arr = zeros(1, length(n_arr));
```

```
    for idx = 1:length(n_arr)
```

```
        if idx-1 >= 1
```

```
            y_prev1 = y_arr(idx-1);
```

```
        else
```

```
            y_prev1 = 0;
```

```
        end
```

```
        if idx-2 >= 1
```

```

        y_prev2 = y_arr(idx-2);
    else
        y_prev2 = 0;
    end
    y_arr(idx) = x_arr(idx) + y_prev1 - c_O*y_prev2;
end
max_arr(m_idx) = max(y_arr);
end
plot(omega_O_arr, max_arr);
xlabel('\omega_O');
ylabel('ymax(\omega_O)');
%% The next two lines will make your plot look better
axis tight;
grid on;

```

Yes, this plot does have a peak. The ω_0 peak is around 0.8. This lines up with our c_0 value. This would suggest the maximum value is provided when ω_0 matches c_0 . The code is essentially the same as what we did earlier in the lab, however we are comparing the maximum value read and putting it into an array. For the plot, we are comparing the ω_0 array with the maximum value array.